

SECURE CODE REVIEW

ReNew Solar Marketing Website

Manual Review of Security Controls & Remediation Changes

CONFIDENTIAL. Contains source excerpts and defect detail. Distribute only to personnel maintaining the codebase. **Note the independence limitation in Section 2.4:** this review examines changes authored during the same engagement.

CODEBASE

renew — ReNew Solar Marketing Site

REVIEW DATE

17 August 2026

BASELINE

a4f4020 (main)

REVIEW TYPE

Manual · change-focused

PRIMARY SCOPE

Remediation diff — 17 files, 174 insertions

SECONDARY SCOPE

Security control design

DEFECTS FOUND

3 — all in remediation code, all fixed

REPORT VERSION

1.0 — Final

1 Executive Summary

This is a manual secure code review. Its primary target is the body of code that has had the least scrutiny and carries the most security weight: **the remediation changes applied on 17 August 2026** — 17 files, 174 insertions, five new files — written specifically to close findings from the VAPT, SAST and DAST reports. Security fixes are among the most dangerous code in any codebase, because they are trusted on the strength of their intent rather than their implementation, and they are frequently shipped without the review ordinary features receive.

PRINCIPAL RESULT

Three defects were found in the remediation code itself. All three were introduced while fixing security findings, all three passed the automated engines cleanly, and all three would have shipped unnoticed. One had a direct functional consequence for the control it was meant to deliver: the `robots.txt` written to close VAPT-06 blocked crawlers from the CSS, JavaScript and fonts the site needs to render — **degrading the very SEO capability it was added to provide.**

All three have been fixed and re-verified. This is the central finding of the review, and the argument for the practice: **fixes need reviewing as rigorously as the code they fix.**

The review also validated a control that had been authored but never actually exercised. The Content Security Policy written for VAPT-01 was, until this review, only a string in a configuration file — asserted to work, never tested. It was applied to the real build and the site loaded under it: **zero policy violations, full hydration, all fonts and styles applied.** That is now evidence rather than expectation.

Findings by severity



What this review adds over the SAST report

DIMENSION	SAST (AUTOMATED + PATTERNS)	SECURE CODE REVIEW (THIS REPORT)
Unit of analysis	The whole tree, as it stands	The change set — what moved, and whether it moved correctly
Question asked	"Does this match a known-dangerous pattern?"	"Does this control actually do what it claims?"
Security controls	Detects their absence	Evaluates their correctness — SCR-01 is a control that existed and was wrong
Design properties	Not assessed	Trust boundaries, fail-safe defaults, complete mediation — Section 4
Consistency	Not assessed	Is a safe pattern applied everywhere it applies? SCR-02 is a lapse in exactly that
Verification	Tool exit status	Control exercised against the real build — Section 5

OVERALL

The codebase's security posture is **good and improving**. The underlying architecture remains the strongest control present, the URL-fragment allow-lists are correctly and deliberately implemented, and the remediation closed real defects. The three problems found here were all in new code, all low-severity, and all fixed — but their existence is the reason this review was worth running, and the reason the recommendation in Section 7.1 is to make change review routine rather than exceptional.

2 Scope & Method

2.1 Primary scope — the remediation change set

FILE	CHANGE	SECURITY PURPOSE
vercel.json	NEW	CSP, framing, MIME, referrer, permissions, COOP/CORP, CORS
src/lib/site.ts	NEW	Canonical origin resolution — carried API-SEC-01
src/app/robots.ts	NEW	Crawler directives — carried SCR-01
src/app/sitemap.ts	NEW	Canonical URL set
public/.well-known/security.txt	NEW	RFC 9116 disclosure channel
src/components/custom-dropdown.tsx	Modified	Required-field enforcement (SAST-02)
src/components/sections/savings-calculator.tsx	Modified	Tariff correctness, clamp notice — carried SCR-02
src/lib/solar.ts	Modified	Input bound clamping — carried SCR-03
src/app/page.tsx	Modified	JSON-LD sink escaping
src/components/sections/footer.tsx	Modified	Removed non-null assertion
solar-module-detail.tsx · manufacturing-detail.tsx	Modified	Type predicates on the URL-fragment allow-lists
contact.tsx · contact-modal.tsx	Modified	Removed the unsupported response-time commitment
next.config.ts · tsconfig.json · package.json	Modified	Dev origin, type coverage, dependency overrides

2.2 Secondary scope — control design

Beyond the diff, the review examined the security-critical paths that remain: the URL-fragment allow-lists, the form validation chain, the calculation domain layer, and the trust boundaries across the application. These are assessed as designs in Section 4, not merely as text.

2.3 Method

1. **Adversarial diff review.** Each change read against the finding it claims to close, asking what it actually does rather than what it intends.
2. **Hypothesis-driven testing.** Where a change looked capable of failing, the failure was *reproduced* rather than asserted. Two hypotheses were confirmed, one refuted — Section 3.4.
3. **Control exercise.** The CSP was applied to the real build and the site loaded under it, with violations captured programmatically.

4. **Consistency audit.** Where a safe pattern exists, is it used everywhere it applies? This is what surfaced SCR-02.
5. **Design principles.** Trust boundaries, fail-safe defaults, complete mediation, defence in depth — assessed in Section 4.
6. **Regression verification.** After each fix, the original control re-tested to confirm the fix did not break what it was protecting.

2.4 Independence limitation

STATED PLAINLY

This review examines code authored during the same engagement, by the same party. Self-review is a genuine control — it found three real defects here that automation missed — but it is not equivalent to independent review. An author is systematically least likely to question the assumption that produced the defect in the first place.

Two mitigations were applied: every claim in this report rests on an executed test rather than on reasoning, and hypotheses were pursued to *disconfirmation* as well as confirmation (Section 3.4 records one hypothesis that testing refuted). Neither substitutes for a second pair of eyes. **Independent review of the remediation diff before it merges is recommended**, and is the first item in Section 7.

3 Findings in the Remediation Code

SCR-01 robots.txt blocked render-critical assets from crawlers**LOW**

CWE	INTRODUCED BY	STATUS	FILE
CWE-1188 — Insecure Default Initialization	VAPT-06 remediation	Fixed & verified	src/app/robots.ts

DESCRIPTION

The `robots.txt` added to close VAPT-06 disallowed `/_next/`, on the reasoning that it holds build artifacts rather than content. That reasoning is wrong: `/_next/static/` is where Next.js places the CSS, JavaScript and font files every page needs to render.

```
// src/app/robots.ts – as written during remediation
rules: {
  userAgent: "*",
  allow: "/",
  // Build artifacts, not content – no value in the index.
  disallow: ["/_next/"], // ← blocks the render-critical assets
}
```

EVIDENCE

```
Disallow: /_next/

# What that rule actually blocks:
22
8

# How much the homepage depends on them:

129 # references in a single page
```

IMPACT

Search engines render pages before indexing them. A crawler obeying this rule fetches the HTML but is refused the stylesheet, the JavaScript and the fonts — so it evaluates a broken, unstyled shell. The consequences are degraded indexing quality, unreliable mobile-friendliness assessment, and distorted Core Web Vitals attribution. Blocking CSS and JS is explicitly advised against by major search engines for exactly this reason.

There is no confidentiality impact — the assets are public either way, and a crawler respecting `robots.txt` is not an adversary. The severity is **Low** and the nature is functional. It earns its place in a security report because of *where* it came from: **a control added to satisfy a security finding, which silently degraded the capability that finding existed to provide**. A fix that is verified only by its own presence is not verified at all.

REMEDATION APPLIED

```
rules: {  
  userAgent: "*",  
  allow: "/",  
  // Deliberately no Disallow. /_next/ holds the CSS, JS and fonts the pages  
  // need to render – blocking it stops crawlers rendering the site and is  
  // explicitly advised against. The sitemap below is what tells crawlers  
  // which URLs are canonical, so the .txt RSC payloads need no rule either.  
}
```

```
# after fix
```

```
Allow: /
```

The sitemap is the correct mechanism for signalling which URLs are canonical, which also addresses the duplicate-content concern the disallow was reaching for — without blocking anything needed to render.

SCR-02 Unsafe object-index lookup reachable via prototype keys

LOW

CWE	INTRODUCED BY	REACHABLE TODAY	STATUS
CWE-1321 — Prototype Pollution (adjacent)	SAST-01 remediation	No — latent	Fixed & verified

DESCRIPTION

The tariff lookup added to fix the calculator used a bare object index with a nullish fallback. A plain object inherits from `Object.prototype`, so keys such as `constructor` or `toString` return an inherited *function* rather than `undefined` — the `?? 8` never fires, and `.toFixed()` is called on a function.

```
// savings-calculator.tsx — as written during remediation
const tariffFor = (state: string) => (STATE_TARIFFS[state] ?? 8).toFixed(2);
```

EVIDENCE

```
# unknown key falls back correctly
THROWS: TypeError: (...).toFixed is not a function
THROWS: TypeError
THROWS: TypeError
THROWS: TypeError
```

IMPACT

Not reachable in the current application. `state` is initialised to a known constant and only ever reassigned from the dropdown's option list, which is derived from the tariff table's own keys. No path exists today by which a prototype key reaches this function; if one did, the result would be an uncaught exception during render — a client-side denial of view of the calculator, not data exposure.

WHY THIS IS WORTH REPORTING DESPITE BEING UNREACHABLE

The DAST report specifically praised this codebase for using `Set.has()` rather than object lookup in its URL-fragment allow-lists, noting that `__proto__` and `constructor` would have behaved very differently under `MAP[hash]`. That was correct — and then remediation introduced, in a neighbouring component, precisely the pattern the report had just identified as the dangerous one.

The defect is the inconsistency, not the crash. A safe pattern that is applied in one place and forgotten in another is a pattern the codebase does not actually have. That is the kind of thing a consistency audit catches and a pattern-matching scanner does not.

REMEDATION APPLIED

```
const FALLBACK_TARIFF = 8;

// hasOwn, not a bare index: a plain-object lookup inherits from
// Object.prototype, so keys like "constructor" or "toString" return a function
// rather than undefined, `?? FALLBACK` never fires, and .toFixed throws. Same
// reasoning as the Set-based allow-lists guarding the URL fragment elsewhere.
const tariffFor = (state: string) =>
  (Object.hasOwn(STATE_TARIFFS, state) ? STATE_TARIFFS[state] : FALLBACK_TARIFF).toFixed(2);
```

8.00	8.00
7.10	8.00
8.00	8.00

`Object.hasOwn` is consistent with the browser baseline the codebase already assumes — `Array.prototype.at()` is used in the modal's focus trap and carries equivalent support requirements.

SCR-03 Clamp notice out of step with the clamp it reports

INFO

CWE	INTRODUCED BY	REACHABLE VIA UI	STATUS
CWE-1077 — Inconsistent Implementation	SAST-04 remediation	No — see 3.4	Fixed & verified

DESCRIPTION

Two functions in `src/lib/solar.ts` had to agree: one clamps the value, the other decides whether to tell the user it was clamped. They disagreed on non-finite input. `calculateSolar` clamps `Infinity` to the maximum, while `isUsageOutOfRange` excluded it via `Number.isFinite` — so an overflowing value was capped with **no notice shown**, which is the precise failure the notice was added to prevent.

```
// as written during remediation
export function isUsageOutOfRange(usage: string | number): boolean {
  const n = Number(usage);
  return Number.isFinite(n) && n > MAX_MONTHLY_UNITS; // excludes Infinity
}

const monthlyUnits = Math.min(MAX_MONTHLY_UNITS, Math.max(0, Number(usage) || 0)); // clamps it
```

EVIDENCE

```
Infinity    true    false    <-- MISMATCH: clamped silently
1e999      true    false    <-- MISMATCH: clamped silently
```

IMPACT

Informational — not reachable through the interface. See Section 3.4: browser testing established that `<input type="number">` rejects `1e999` and `Infinity` outright, so the value never reaches component state. The mismatch is real in the library but cannot currently be triggered by a user.

It is reported because `calculateSolar` and `isUsageOutOfRange` are **exported**, and the API control specification recommends the eventual server-side endpoint reuse this domain layer. A server-side caller has no `type="number"` input filtering on its behalf, and would hit the mismatch directly.

REMEDATION APPLIED

```
export function isUsageOutOfRange(usage: string | number): boolean {
  const n = Number(usage);
  if (Number.isNaN(n)) return false; // empty / non-numeric yields 0, not a cap
  return n > MAX_MONTHLY_UNITS;     // Infinity > MAX is true – matches the clamp
}
```

```
true    true
true    true
```

ONE CASE REVIEWED AND DELIBERATELY LEFT

The same consistency check flags `-500` : it is clamped, to `0` , without a notice. This was considered and **accepted**. The notice is upper-bound specific by design, and a negative input renders a visible `₹0` — self-evidently not a real estimate, unlike the plausible-looking ₹288-trillion figure the original finding was about. The input also carries `min={1}` , so the browser blocks it on submit. Adding a second notice would be over-engineering for no gain in honesty.

3.4 A hypothesis that testing refuted

RECORDED BECAUSE NEGATIVE RESULTS MATTER

SCR-03 was expected to be *reachable*. The reasoning looked sound: `1e999` is valid scientific notation, an `<input type="number">` accepts scientific notation, and `Number("1e999")` overflows to `Infinity`. That would have made the silent clamp a live user-facing defect and raised the severity to Low.

Testing in a real browser refuted it. The input rejects values that overflow to infinity, so the field empties and the calculator shows zero:

```
1e999      ""  
Infinity   ""
```

The finding was therefore downgraded from Low to Informational. This is recorded rather than quietly dropped because a review that only reports confirmed hypotheses gives no signal about how hard it looked — and because the same instinct that would have let this be reported as Low, unverified, is the instinct that produced the defects in Section 3 in the first place.

4 Security Control Design Review

Beyond individual defects, the review assessed whether the security controls are well-placed — a correct control in the wrong layer fails as surely as an incorrect one.

4.1 Trust boundaries

BOUNDARY	CONTROL	ASSESSMENT
URL fragment → component state	CORRECT	Closed-set allow-list via <code>Set.has()</code> , now behind a type predicate so removing the guard is a compile error. The strongest control in the codebase.
Form input → component state	CORRECT	Values are held in state and rendered as text by React. No sink is reachable.
Form input → calculation	CORRECT	Clamped in the domain layer rather than the component, so every caller inherits the bound. Correct placement.
Build env → published output	WEAK POINT	The boundary that failed twice — API-SEC-01 and SCR-01 both live here. Now validated in <code>site.ts</code> , but this is the least-defended boundary in the codebase.
Static data → JSX attributes	CORRECT	All 49 dynamic <code>href</code> / <code>src</code> values originate in module constants.
Component data → raw HTML sink	CORRECT	Single JSON-LD sink, static input, now escaped defensively.

THE PATTERN WORTH NAMING

Three of the defects found across this engagement's remediation — API-SEC-01, SCR-01, and arguably SCR-03 — share a root cause: **build-time inputs and build-time generated output were not treated as a trust boundary**. Code that runs at build time feels safe because it runs on a developer's machine, from values a developer set. But it writes files that are published to every visitor, and it fails silently — every one of these defects produced a successful build with exit code 0.

The generalisable control: **validate configuration at the point of use, and assert on the generated artifact, not the generating code**. Section 7.1 proposes a build-time check that would have caught both API-SEC-01 and SCR-01 automatically.

4.2 Fail-safe defaults

SITUATION	BEHAVIOUR	ASSESSMENT
Unknown URL fragment	Falls back to default tab	FAIL-SAFE
Unknown / prototype state key	Falls back to base tariff	FAIL-SAFE (after SCR-02)
Invalid <code>SITE_URL</code>	Falls back to known-good origin	FAIL-SAFE (after API-SEC-01)
Out-of-range usage	Clamped, and the user is told	FAIL-SAFE (after SCR-03)
Non-numeric usage	Yields 0	FAIL-SAFE
Missing <code>headingHref</code>	Renders plain heading	FAIL-SAFE
Form submission failure	Reports success	FAIL-OPEN — the one violation

The codebase is consistently fail-safe with a single exception, and it is the open High finding (VAPT-02 / SAST-03 / DAST-02): the form's success state is set unconditionally rather than derived from a result. That is a fail-open default. The client deferred the endpoint, so it stands — but it should be corrected as part of building the endpoint, not after.

4.3 Complete mediation

Is each control applied on *every* path, or only the obvious one?

CONTROL	COVERAGE	ASSESSMENT
Fragment allow-list	2 OF 2	Both detail components; identical implementation
Required-field enforcement	ALL	Fixed in the shared <code>CustomDropdown</code> , so all three consumers inherit it — correct placement
Input clamping	ALL	In the domain layer, not the component
Tariff lookup guard	ALL	Single function, all call sites route through it
Security headers	PENDING	<code>/(.*)</code> covers every route — but unverified until deployed
Response-time promise removal	3 OF 3	Verified absent from rendered output

Fixing the required-field defect inside the shared component rather than at each of the three call sites is the correct structural choice, and worth noting as a positive: it makes the control impossible to forget at a future fourth call site.

4.4 Defence in depth

THREAT	LAYERS PRESENT
XSS	No sinks · React auto-escaping · JSON-LD escape · CSP (pending deploy) — 4 LAYERS
Clickjacking	X-Frame-Options · frame-ancestors — 2, BOTH PENDING DEPLOY
Fragment injection	Allow-list · type predicate · no sink downstream — 3 LAYERS
Bad calculation input	HTML min/max · domain clamp · user notice — 3 LAYERS
Supply chain	Overrides · bun audit clean — 2, NO CI GATE YET

Clickjacking is the thinnest: both layers are the same mechanism (response headers) delivered by the same file, so a `vercel.json` that fails to deploy removes the entire defence. There is no JavaScript frame-busting fallback — noted in DAST-01 and still true. This is an acceptable design given the site holds no session, but it means **deployment verification is the whole control**, not a formality.

5 Control Verification

5.1 The CSP, exercised for the first time

The Content Security Policy authored for VAPT-01 had never been run. A policy that blocks the application's own resources is worse than none — it breaks the site and gets rolled back, leaving the finding open. It was therefore applied to the real production build via a local server replaying the exact `vercel.json` header set, and the site loaded under it.

```
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline'; ...
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: accelerometer=(), autoplay=(self), camera=(), ...
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-site
Access-Control-Allow-Origin: https://re-new-mocha.vercel.app
```

Violations were captured programmatically rather than read off the console:

```
"cspViolationCount": 0,

# and the page is genuinely functional, not a surviving shell:

"hydrated": true,           # client components mounted
"cssApplied": true,
"fontFacesLoaded": 9,
```

RESULT

The CSP is validated. Zero violations, full hydration, styles and fonts applied, both forms present. This is achievable because the site loads no third-party resources — confirmed in the VAPT report and still true. `'unsafe-inline'` remains necessary for Next.js's inline bootstrap; it can be tightened to nonces later, but the directives carrying the security weight here — `frame-ancestors 'none'`, `object-src 'none'`, `base-uri 'self'` — are already strict.

One console error appeared: `ERR_SSL_PROTOCOL_ERROR`. This is an artifact of `upgrade-insecure-requests` rewriting `http://` subresources to `https://` on a plain-HTTP local test server. It cannot occur in production, which is HTTPS-only with HSTS preloaded. It is *not* a CSP violation — the violation counter is authoritative, and it recorded zero.

5.2 Regression verification after each fix

Every fix in Section 3 was followed by re-testing the control it touched, since a fix that breaks the thing it protects is a net loss.

FIX	REGRESSION TEST	RESULT
SCR-02 — <code>hasOwn</code> guard	Four states re-selected through the real UI	4 DISTINCT TARIFFS — 11.00 / 5.20 / 7.10 / 8.00
SCR-03 — notice logic	Overflow entered; then a valid value	NOTICE FIRES AND CLEARS
SCR-01 — robots rule	Regenerated and inspected	NO DISALLOW; SITEMAP INTACT
All	<code>tsc</code> · ESLint · <code>next build</code>	CLEAN — 9 static routes
All	Browser console during interaction	ZERO MESSAGES

5.3 Items reviewed and accepted without change

ITEM	REASONING
Negative usage clamped without notice	Yields a visible <code>₹0</code> — self-evidently not an estimate, unlike a plausible large figure. <code>min={1}</code> blocks it on submit.
<code>aria-hidden</code> on the validation input	Correct: the combobox above carries the accessible name and is the sole keyboard target. Exposing both would produce a duplicate announcement.
Dependency <code>overrides</code> block	Necessary — the advisories were unreachable via parent ranges. Carries a maintenance cost: it can mask legitimate future resolutions. Revisit at each major upgrade.
<code>TODO(batch-4-followup)</code> markers left in place	Correct. They are the tracking record for the deferred endpoint; removing them while the defect stands would hide it.
<code>'unsafe-inline'</code> in <code>script-src</code>	Required by the framework's inline bootstrap. Documented rather than removed; nonce-based tightening is a later refinement, not a defect.

6 Secure Coding Practice Compliance

Assessed against the OWASP Secure Coding Practices checklist, restricted to the sections that apply to a static client-rendered application. Sections covering authentication, session management, database security and server configuration are omitted — there is no server.

PRACTICE AREA	STATUS	EVIDENCE
Input validation	COMPLIANT	Allow-lists on the only untrusted input; clamping in the domain layer; required-field enforcement fixed in the shared component
Output encoding	COMPLIANT	React auto-escaping throughout; the single raw sink escapes <code><</code>
Error handling	COMPLIANT	Fail-safe fallbacks everywhere except the deferred form submission; no stack traces or internal detail surfaced
Logging	COMPLIANT	Zero console output in production; no PII logged (nothing is logged)
Data protection	PARTIAL	No client-side storage, no cookies, HTTPS enforced with HSTS. Privacy notice deferred — becomes non-compliant when the endpoint ships
Communication security	COMPLIANT	TLS 1.3, legacy protocols refused, HSTS preloaded, no mixed content
System configuration	PENDING	Header set authored and locally validated; unverified in production until deployed
File management	COMPLIANT	No upload capability; published assets reviewed and intentional
Memory / resource management	COMPLIANT	All listeners, observers and timers released — verified in the SAST review
General coding practices	COMPLIANT	Strict TypeScript with zero escapes; dependencies patched; no dynamic code evaluation

6.1 Findings summary

ID	SEVERITY	FINDING	STATUS	ORIGIN
SCR-01	LOW	robots.txt blocked render-critical assets	FIXED	VAPT-06 fix
SCR-02	LOW	Unsafe object-index lookup via prototype keys	FIXED	SAST-01 fix
SCR-03	INFO	Clamp notice out of step with the clamp	FIXED	SAST-04 fix
SCR-04	INFO	CSP has no violation reporting endpoint	OPEN	VAPT-01 fix
SCR-05	INFO	Single-origin CORS without <code>Vary: Origin</code>	OPEN	VAPT-05 fix
SCR-06	INFO	Sitemap <code>lastmod</code> churns on every build	OPEN	VAPT-06 fix

6.2 The three open informational items

SCR-04 — no CSP reporting. The policy has no `report-to` or `report-uri`, so violations in production are invisible. The policy was validated locally against today's build, but a future change introducing a blocked resource will surface as a silently broken feature rather than an alert. Adding a reporting endpoint — or at minimum a `Content-Security-Policy-Report-Only` rollout before enforcing — converts that into a signal.

SCR-05 — `Vary: Origin`. The CORS header is now a single literal origin rather than a wildcard, which is the improvement VAPT-05 called for. Strictly, any response whose headers depend on the request `Origin` should carry `Vary: Origin` so shared caches key correctly. Here the value is a constant rather than reflected, so no cache-poisoning risk arises — the note applies if the header ever becomes dynamic.

SCR-06 — `sitemap lastmod`. Every URL is stamped with the build time, so a deploy that changes one page advertises all six as modified. Crawlers learn to discount an always-changing `lastmod`. Not a security issue; noted because it slightly undermines the SEO control VAPT-06 delivered. A content-derived date would be more truthful.

7 Recommendations

7.1 Process — the durable fixes

#	RECOMMENDATION	RATIONALE
1	Independent review of the remediation diff before merge	Directly addresses the limitation in Section 2.4. Three defects were found by self-review; the honest expectation is that an independent reviewer finds more.
2	Assert on build output, not just build success	Would have caught both API-SEC-01 and SCR-01 automatically. Both produced exit code 0. See 7.2.
3	Unit tests for <code>lib/solar.ts</code>	Still the highest-leverage single control. SAST-01, SAST-04, SCR-02 and SCR-03 all live in or beside this 60-line pure function, which has no tests.
4	Deploy, then re-verify the headers	Three findings across the VAPT and DAST reports remain formally open purely for want of a deployment. The clickjacking exploit should be assumed live until the PoC is re-run.
5	CI gate on the existing clean baseline	Semgrep, ESLint, <code>tsc</code> and <code>bun audit</code> all pass at zero. Making them blocking costs nothing today and prevents regression.
6	Treat build-time config as untrusted input	The trust boundary that failed twice. Validate at the point of use, as <code>site.ts</code> now does.

7.2 The build-output assertion, concretely

Both build-time defects shipped a valid-looking build. A short post-build check would have failed the build instead:

```
// scripts/verify-build.ts – run after `next build`
import { readFileSync } from "node:fs";

const robots = readFileSync("out/robots.txt", "utf8");
const sitemap = readFileSync("out/sitemap.xml", "utf8");
const fail = (m: string) => { console.error(`build verification failed: ${m}`); process.exit(1); };

// SCR-01: never block the assets pages need to render
if (/^Disallow:\s*\/_next/mi.test(robots)) fail("robots.txt blocks /_next/ (render-critical assets)");

// API-SEC-01: the sitemap must be parseable, or crawlers silently ignore it
if (/&(?!amp;|lt;|gt;|quot;|apos;)|<script/i.test(sitemap)) fail("sitemap.xml contains unescaped markup");

// Both files must name the same canonical origin
const origin = sitemap.match(/<loc>(https?:\/\/\[^\]]+)/)?.[1];
if (!origin || !robots.includes(origin)) fail("robots.txt and sitemap.xml disagree on origin");

console.log("build verification passed");
```

Wire as `"build": "next build && bun scripts/verify-build.ts"`. Roughly twenty lines, and it closes the weakest trust boundary identified in Section 4.1 permanently.

7.3 Code — remaining items

#	ID	ACTION	PRIORITY
1	SCR-04	Add CSP reporting, or roll out report-only first	WITH DEPLOY
2	SCR-06	Derive sitemap <code>lastmod</code> from content, not build time	OPPORTUNISTIC
3	SCR-05	Add <code>Vary: Origin</code> if the CORS value ever becomes dynamic	CONDITIONAL
4	—	Update the three hardcoded origins together on the domain move: <code>vercel.json</code> , <code>security.txt</code> , <code>NEXT_PUBLIC_SITE_URL</code>	ON MIGRATION
5	—	Correct the fail-open success state when the form endpoint is built	WITH ENDPOINT

8 Appendix

8.1 Where this report sits

REPORT	VERSION	METHOD	DISTINCT CONTRIBUTION
VAPT	1.1	Black-box + source	External surface; established the static-export architecture
SAST	1.1	Automated + manual	3 engines at zero findings; all defects from manual review
DAST	1.1	Runtime	Confirmed clickjacking exploit; runtime proof of the form defect
API	1.0	Endpoint-focused	Verified API absence; control spec for the deferred endpoint
SCR	1.0	Manual, change-focused	Reviewed the fixes themselves — found 3 defects the other four could not, because the code did not exist when they ran

8.2 Cumulative position

STATUS	COUNT	DETAIL
Closed and verified	15	Including all three found in this review
Fixed, pending deployment	3	Clickjacking and the header set — assume still exploitable
Mitigated, open by client decision	1	Form endpoint — the highest-priority open item
Accepted by client	3	Privacy notice (coupled to the above), category control, PDF metadata
Open informational	3	SCR-04, SCR-05, SCR-06

8.3 Method notes

- **Diff review** against `a4f4020` : 17 files, 174 insertions, 33 deletions, plus five new files.
- **Hypothesis testing** in Node for pure logic; in an instrumented Chromium for anything depending on browser behaviour — which is what refuted the SCR-03 reachability hypothesis.
- **CSP verification** via a local server replaying the exact `vercel.json` header set against the production build, with violations captured through the `securitypolicyviolation` event.
- **Regression testing** after each fix, through the real UI rather than by inspection.

8.4 Limitations

- **Not independent.** Section 2.4. The single most valuable next step is a second reviewer on the same diff.

- **Deploy-dependent controls unverified in production.** Local header replay is a strong proxy but is not the Vercel edge.
- **Point-in-time**, reflecting `a4f4020` plus remediation as of 17 August 2026.
- **No test suite to review.** The codebase has no automated tests, so this review could assess neither their coverage nor their correctness — which is itself the basis for recommendation 3.

END OF REPORT

ReNew Solar Marketing Website — Secure Code Review v1.0 — 17 August 2026. Scope: 17 changed files, 5 new files, 174 insertions. 6 findings: 0 Critical, 0 High, 0 Medium, 2 Low, 4 Informational. 3 defects found in remediation code, all fixed and re-verified. CSP validated against the real build at 0 violations.